CSE 815: Machine Learning

Week 2, Part 1 — Mathematical Foundations

Rokan Uddin Faruqui
Professor
Department of Computer Science & Engineering
University of Chittagong, Chittagong

CSE, CU, 2026

Course & Lecture Information

Course Details

Code: CSE 815
Credits: 3 (Theory)
Type: Elective – Data Science
Pre-req: CSE 221 (Data Structures), CSE 225 (Algorithms), CSE 235 (Probability & Stats)

Today

Week / Part: 2 of 13 | Part 1 of 2
CLO: CLO3
PLO: PLO(b)

CLO3

"Explain the mathematical principles (optimization, probability, and statistical inference) underlying learning algorithms"

Part 1 Covers

- Linear Algebra: vectors, matrices, matrix operations, eigenvalues, SVD
- Probability: Bayes' theorem, distributions, MLE, MAP
- Optimization: gradient descent, convexity, loss functions
- How these tools appear in ML algorithms

Lecture Agenda

1. Why Mathematics? The Three Pillars of ML
2. Linear Algebra
3. Probability and Statistics
4. Optimization
5. Student Assessment Pool

Why Mathematics? The Three Pillars of ML

Machine Learning = Linear Algebra + Probability + Optimization

The Three Pillars

1. **Linear Algebra**: Represents data, models, and transformations.
   - Data points as vectors $\mathrm{x} \in \mathbb{R}^d$
   - Model parameters as matrices/vectors W, b
   - Operations: matrix multiplication, dot products, projections
2. **Probability**: Handles uncertainty, noise, and inference.
   - Data generation process: $p(\mathrm{x}, y)$
   - Model likelihood: $p(y \mid \mathrm{x}, \theta)$
   - Bayesian reasoning: $p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta) p(\theta)$
3. **Optimization**: Finds the best model parameters.
   - Minimise loss $L(\theta)$ w.r.t. θ via gradient descent $\theta \leftarrow \theta - \alpha \nabla_\theta L$
   - Convex vs. non-convex landscapes

The Key Insight

Every ML algorithm is, at its core, a **composition** of these three mathematical frameworks. Understanding them deeply is essential to understand *why* algorithms work (and why they fail).

Linear Algebra

Vectors: Data Points in Space

Definition

A vector $\mathbf{v} \in \mathbb{R}^d$ is an ordered list of d real numbers:

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_d \end{pmatrix} = (v_1, v_2, \ldots, v_d)^T$$

It represents a point in d-dimensional space.

Key Operations

Operation	Definition
Addition	$\mathbf{u} + \mathbf{v} = (u_1 + v_1, \ldots, u_d + v_d)$
Scalar multiplication	$c\mathbf{v} = (cv_1, \ldots, cv_d)$
Dot product	$\mathbf{u}^T\mathbf{v} = \sum_{i=1}^{d} u_i v_i$
Norm (length)	$\|\mathbf{v}\| = \sqrt{\mathbf{v}^T\mathbf{v}} = \sqrt{\sum_i v_i^2}$
Distance	$\|\mathbf{u} - \mathbf{v}\|$

ML Application

A data sample with d features is a vector. Model parameters are also vectors (e.g., weights $\mathbf{w}$ in linear regression). The prediction is $\hat{y} = \mathbf{w}^T\mathbf{x}$ — a dot product.

Matrices: Linear Transformations

Definition

A matrix $A \in \mathbb{R}^{m \times n}$ is a rectangular array with m rows and n columns.

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Key Operations

Operation	Definition
Matrix-vector product	$Ax \in \mathbb{R}^m$, $(Ax)_i = \sum_j A_{ij} x_j$
Matrix-matrix product	AB, $(AB)_{ij} = \sum_k A_{ik} B_{kj}$
Transpose	$(A^T)_{ij} = A_{ji}$
Inverse	$A^{-1}A = AA^{-1} = I$
Trace	$\mathrm{tr}(A) = \sum_i A_{ii}$

ML Application: Linear Regression

The prediction for all n samples is $\hat{y} = Xw$, where $X \in \mathbb{R}^{n \times d}$ is the design matrix (features), $w \in \mathbb{R}^d$ are the weights. The normal equations: $w = (X^T X)^{-1} X^T y$.

Eigenvalues and Eigenvectors

Definition

For a square matrix $A \in \mathbb{R}^{n \times n}$, a non-zero vector v is an **eigenvector** if:

$$Av = \lambda v$$

where λ is the corresponding **eigenvalue**.

Why They Matter in ML

- **PCA**: The principal components are the eigenvectors of the covariance matrix.
- **Spectral clustering**: Uses eigenvectors of the graph Laplacian.
- **Convergence analysis**: Learning dynamics depend on eigenvalues of the Hessian.
- **Matrix square roots**: Used in Mahalanobis distance.

PCA Derivation

The first principal component v_1 is the eigenvector of the covariance matrix $\mathbf{\Sigma}$ with the largest eigenvalue λ_1. It maximises variance of projected data:

$$v_1 = \arg \max_{\|v\|=1} v^T \mathbf{\Sigma} v$$

Singular Value Decomposition (SVD)

Theorem

Any matrix $A \in \mathbb{R}^{m \times n}$ can be factorised as:

$$A = U\Sigma V^T$$

where:

- $U \in \mathbb{R}^{m \times m}$ is orthogonal (columns = left singular vectors)
- $\Sigma \in \mathbb{R}^{m \times n}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$
- $V \in \mathbb{R}^{n \times n}$ is orthogonal (columns = right singular vectors)

ML Applications

- **Dimensionality reduction**: Truncated SVD keeps top k singular values (PCA via SVD).
- **Low-rank approximation**: $A \approx U_k \Sigma_k V_k^T$ (used in collaborative filtering).
- **Matrix pseudoinverse**: $A^+ = V\Sigma^+ U^T$ (used in linear regression when $n < d$).
- **Latent semantic analysis**: Document-term matrix SVD for topic modelling.

Probability and Statistics

Probability Essentials for ML

Key Concepts

- **Random variable**: X — a variable whose value is subject to randomness.
- **Probability distribution**: $p(X)$ — assigns probabilities to outcomes.
- **Joint distribution**: $p(X, Y)$ — probability of both events.
- **Conditional probability**: $p(Y \mid X) = \frac{p(X,Y)}{p(X)}$ — probability of Y given X.
- **Expectation**: $\mathbb{E}[f(X)] = \sum_x p(x)f(x)$ (discrete) or $\int p(x)f(x)dx$ (continuous).
- **Variance**: $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$ — measure of spread.
- **Covariance**: $\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$ — measure of linear dependence.

ML Context

The data generation process is modelled as sampling independent and identically distributed (IID) from an unknown distribution $p(x, y)$. Learning aims to infer properties of this distribution from observed samples.

Bayes' Theorem: The Core of Probabilistic ML

Statement

$$p(A \mid B) = \frac{p(B \mid A)p(A)}{p(B)}$$

In Machine Learning (Bayesian Inference)

Let θ be model parameters, $\mathcal{D}$ be data. Then:

$$\underbrace{p(\theta \mid \mathcal{D})}_{\text{posterior}} = \frac{\overbrace{p(\mathcal{D} \mid \theta)}^{\text{likelihood}}\overbrace{p(\theta)}^{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{evidence (marginal likelihood)}}}$$

- **Prior**: Belief about θ before seeing data.
- **Likelihood**: How probable the data is under a given θ.
- **Posterior**: Updated belief after seeing data.
- **Evidence**: $p(\mathcal{D}) = \int p(\mathcal{D} \mid \theta)p(\theta)d\theta$, acts as normalising constant.

Maximum Likelihood Estimation (MLE)

The Idea

Choose parameters θ that **maximise the probability** of the observed data:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} p(\mathcal{D} \mid \theta)$$

For independent and identically distributed (IID) data $\mathcal{D} = \{\mathrm{x}_i, y_i\}_{i=1}^n$:

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{n} p(y_i \mid \mathrm{x}_i, \theta)$$

It is common to maximise the **log-likelihood** (since log is monotonic):

$$\ell(\theta) = \log p(\mathcal{D} \mid \theta) = \sum_{i=1}^{n} \log p(y_i \mid \mathrm{x}_i, \theta)$$

Linear Regression as MLE

Assume $y \mid \mathrm{x}, \mathrm{w} \sim \mathcal{N}(\mathrm{w}^T \mathrm{x}, \sigma^2)$. Then:

$$\ell(\mathrm{w}) = -\frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - \mathrm{w}^T \mathrm{x}_i)^2 + \text{constant}$$

Maximising ℓ is equivalent to minimising the sum of squared errors — **least squares**.

Optimization

The Optimization Problem in ML

General Formulation

We want to find:

$$\theta^* = \arg\min_{\theta \in \Theta} L(\theta)$$

where:

- θ — model parameters (e.g., weights in neural networks)
- $L(\theta)$ — loss function (empirical risk or regularised objective)
- Θ — feasible parameter space

Common Loss Functions

Task	Loss Function
Regression (MSE)	$L(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
Classification (Cross-entropy)	$L(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$
SVM (Hinge)	$L(\theta) = \frac{1}{n}\sum_{i=1}^{n}\max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$
L2 Regularisation	$L(\theta) = \text{loss} + \lambda\|\theta\|_2^2$
L1 Regularisation	$L(\theta) = \text{loss} + \lambda\|\theta\|_1$

Gradient Descent: The Workhorse of ML

The Algorithm

For differentiable L, repeat until convergence:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta L(\theta_t)$$

where $\alpha > 0$ is the **learning rate** (step size).

Intuition

- The gradient ∇L points in the direction of **steepest ascent** of L.
- Moving opposite to the gradient reduces L (steepest descent).
- Small steps ensure we don't overshoot the minimum.
- For convex L, gradient descent converges to the global minimum.

Variants

- **Batch GD**: Uses all training data per step.
- **Stochastic GD (SGD)**: Uses one random sample per step (fast, noisy).
- **Mini-batch GD**: Uses a small batch (e.g., 32–256) — most common in deep learning.

Convexity: The Landscape Matters

Definition (Convex Function)

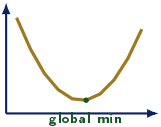
A function $f : \mathbb{R}^d \to \mathbb{R}$ is convex if for any x, y and $t \in [0, 1]$:

$$f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y)$$

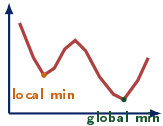
Geometrically: the line segment between any two points lies above the graph.

Why Convexity Matters

- Any local minimum is also a global minimum.
- Gradient descent (and variants) converges to the global optimum.
- Many classic ML problems are convex: linear regression, logistic regression, SVM (with proper formulation).
- Deep learning loss landscapes are **non-convex** — many local minima, saddle points. This makes optimization harder and more heuristic.



Convex (bowl-shaped)



Non-convex (many local minima)

The Chain Rule and Backpropagation

Chain Rule (Univariate)

For $f(g(x))$:

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Multivariate Chain Rule

For a composition $z = f(y)$, $y = g(x)$:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

where $\cdot$ is matrix multiplication (Jacobians).

Why It's Central to ML

Neural networks are **compositions** of many layers:

$$\hat{y} = f_L(f_{L-1}(\dots f_1(x)\dots))$$

The loss L depends on the output, which depends on all parameters through the chain of functions. **Backpropagation** is just the chain rule applied efficiently:

- Forward pass: compute all activations.
- Backward pass: compute gradients layer by layer using the chain rule.

Student Assessment Pool

Assessment—Multiple Choice Questions: Set A *CLO3*

Instructions

Select the single best answer. Answers are **bolded** for instructor reference; present without bold to students.

Q1 In Bayes' theorem $p(A \mid B) = \frac{p(B|A)p(A)}{p(B)}$, $p(A)$ is called the:

(a) Likelihood (b) **Prior** (c) Posterior (d) Evidence

Q2 Which optimization algorithm updates parameters as $\theta_{t+1} = \theta_t - \alpha \nabla_\theta L(\theta_t)$?

(a) Newton's Method (b) **Gradient Descent** (c) Coordinate Descent (d) Conjugate Gradient

Instructor Notes

Q1: Students often confuse prior with posterior. Remind that prior is before seeing data, posterior is after.

Q2: Gradient descent is the most common; Newton's method uses second-order information (Hessian).

Assessment—Multiple Choice Questions: Set B *CLO3*

3. The trace of a square matrix is defined as:
   (a) The product of its eigenvalues (b) **The sum of its diagonal elements** (c) The determinant of the matrix (d) The rank of the matrix

4. Maximum Likelihood Estimation (MLE) finds parameters that:
   (a) Minimise the prior probability (b) **Maximise the likelihood of the observed data** (c) Minimise the posterior probability (d) Maximise the model complexity

Instructor Notes

Q3: Trace is simple but often forgotten; useful in matrix calculus.
Q4: MLE is a frequentist approach; MAP includes prior.

Assessment—True / False

CLO3

Instructions

State True or False. Award full marks only if the student provides a **one-sentence justification**.

Statement	Answer
1. Gradient descent is guaranteed to find the global minimum for any function.	**False**
2. The dot product of two vectors measures their similarity (angular).	**True**
3. MAP estimation is equivalent to MLE when the prior is uniform.	**True**
4. For a convex function, every local minimum is also a global minimum.	**True**

Justification Model Answers

1: Only for convex functions; non-convex functions can have local minima.
2: $u^T v = \|u\|\|v\| \cos\theta$ — it tracks the angle, but also scales with both magnitudes; normalising gives *cosine similarity*.
3: MAP = MLE + log prior; if prior is constant, log prior term is constant.
4: By definition, every local minimum of a convex function is global.

Assessment—Descriptive Questions

CLO3

Instructions

Answer each question in **100–150 words**. Use equations where appropriate.

D1. State Bayes' theorem. Define prior, likelihood, and posterior in the context of Bayesian machine learning.

D2. What is gradient descent? Explain the role of the learning rate and the trade-offs between large and small values.

D3. Explain the difference between MLE and MAP estimation. When would you prefer one over the other?

Instructions

Answer in **200–300 words**. Show derivations step by step.

A1. **Derivation Exercise:**
For linear regression $y = \mathbf{w}^T\mathbf{x} + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, derive the log-likelihood and show that MLE is equivalent to minimising mean squared error.

A2. **Optimization:**
Consider the loss function $L(\theta) = (\theta - 3)^2$. Show that it is convex. Perform two iterations of gradient descent starting from $\theta_0 = 0$ with learning rate $\alpha = 0.1$. What is the final θ after 100 iterations?

Textbook & Reading References

Primary Textbooks for Week 2

- **[Bishop]** Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. **Appendix (Linear Algebra), Ch. 1.2 (Probability)**
- **[Murphy]** Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press. **Ch. 2 (Probability), Ch. 4 (Optimization)**
- **[Hastie]** Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer. **Ch. 2 (Overview)**
- **[Goodfellow]** Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. **Ch. 4 (Numerical Computation)**

Supplementary Reading

- Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2019). *Mathematics for Machine Learning*. Cambridge University Press. *(Free PDF available)*
- Boyd, S. & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge. *(Ch. 1-3 for convexity)*

End of Week 2, Part 1

CSE 815: Machine Learning

"Mathematics is the language in which God has written the universe."

— Galileo Galilei

Next Lecture: Week 2, Part 2

Bias-Variance Decomposition · Normal Equations · Ridge & Lasso · Matrix Calculus

Reading: **[Bishop]** Ch. 3.1–3.2 | **[Hastie]** Ch. 3